Geometry, Noise, and Disturbances: A Diagnostic Study of Model-Based Reinforcement Learning for Data-Efficient Robotic Throwing

AR525 Group 3: Aarya Agarwal, Bhumika Gupta, Rishang Yadav, Yajesh Chandra
B23390, B23036, B23173, B23185
Indian Institute of Technology Mandi

Abstract

Robotic throwing offers a path to fast, long-range manipulation in industrial settings, but learning reliable release policies demands data efficiency: physical robots tolerate only tens of trials before wear or safety constraints intervene. MC-PILOT Turcato et al. [2025] addresses this via a Gaussian-process (GP) dynamics model and a single-shot radial-basis-function (RBF) speed policy optimised with Monte Carlo policy gradients, achieving near-perfect target coverage in roughly ten throws on a Franka Panda arm.

We reproduce MC-PILOT in an independent ballistic simulator, diagnose four non-obvious silent failure modes in the original codebase, and conduct five experimental studies to characterise the algorithm's robustness and portability. From this diagnostic process we derive two portable design rules: a *lengthscale-to-range scaling rule* ($\ell_s \approx 0.15 \times \text{target_range}$) that prevents RBF blindness in narrow target domains, and a *stratified exploration policy* that provides seed-robust GP initialisation without additional hardware trials. We achieve 100% hit rates across five release heights ($z \in \{0.0, 0.5, 1.0, 1.5, 2.0\}$ m) and demonstrate arm-agnostic portability across three robot platforms (Franka Panda, KUKA iiwa7, xArm6) through a decoupled architecture that separates RL from arm kinematics. We prove a *zero-mean noise invariance theorem*—symmetric additive noise does not shift the optimal policy mean, only biased noise is learnable—and validate it empirically: a noise-aware policy achieves 100% hit rate versus 0% for a naive baseline under 20% velocity slip. Finally, we present a wind robustness study revealing a counterintuitive curse-of-dimensionality effect: blind GP policies outperform explicit wind-conditioned policies across all three wind regimes in the low-data budget, and a closed-loop vision pipeline (OpenCV HSV segmentation and YOLOv8n detection) that enables autonomous bin targeting without pre-specified target coordinates.

1 Introduction

Robotic throwing extends the effective reach of a manipulator far beyond its kinematic workspace, enabling rapid sorting, bin-to-bin transfer, and long-range placement without additional actuators. Industrial pick-and-throw cells can double throughput compared with pick-and-place because the robot releases the object at high speed and immediately returns for the next grasp while the projectile is still in flight Turcato et al. [2025]. Despite this appeal, practical deployment is bottlenecked by *data efficiency*: learning a reliable release policy on physical hardware must converge in tens of trials rather than thousands, because each trial incurs wear, reconfiguration overhead, and safety risk. Standard model-free RL methods (*e.g.*TossingBot Zeng et al. [2020]) require orders of magnitude more samples than this budget permits, motivating model-based approaches that explicitly represent uncertainty and plan through it.

MC-PILOT Turcato et al. [2025] addresses this challenge by combining MC-PILCO Mitsubishi Electric Research Laboratories [2022]—a Gaussian-process (GP) dynamics model with Monte Carlo policy gradients—with two throwing-specific modifications. First, the policy is reduced to a single-shot speed map $\pi_\theta(\mathbf{P})$ that fires exactly once at t=0 given the target position $\mathbf{P}$, eliminating the need for closed-loop feedback during flight. Second, gripper release delay is estimated by Bayesian Optimisation, compensating for systematic timing errors that

would otherwise corrupt every trajectory. The augmented state $\tilde{\mathbf{x}} = [\mathbf{x}, \mathbf{P}]$ lets the GP learn geometry-conditioned corrections, and the RBF policy network enables smooth interpolation across the target workspace. On a Franka Panda arm with a Gazebo/ROS simulator, MC-PILOT reaches near-100% target coverage in approximately ten physical throws.

Despite these results, several practical gaps remain unexplored. **(1) Geometry sensitivity.** The published hyperparameters are calibrated to a single release height and target range; naïve transfer to different heights silently degrades performance. **(2) Arm portability.** The paper is locked to a single robot platform; it is unknown whether the algorithm's data-efficiency transfers to other manipulators. **(3) Noise structure.** Physical robots exhibit velocity slip, timing jitter, and gripper compliance whose statistical structure differs from zero-mean additive noise; the learnability of each noise type is not characterised. **(4) Environmental disturbances.** Outdoor or warehouse environments introduce wind, yet the algorithm has not been tested under ambient airflow. **(5) Perception.** Deployment requires autonomous bin detection; the original work pre-specifies target coordinates rather than detecting them visually. **(6) Reproducibility.** The original implementation requires Gazebo and ROS, limiting accessibility; the algorithm's core physics is ballistic and admits a lightweight numpy simulation.

Contributions.

1. **Independent simulation reproduction.** A self-contained ballistic simulator and PyBullet visualiser that replicate MC-PILOT without Gazebo or ROS, with all adaptation decisions documented.

2. **Four diagnosed failure modes.** Systematic identification and correction of four non-obvious silent failure modes: broken gradient path, post-landing GP hallucination, cost-lengthscale saturation, and simulation-horizon unit mismatch.

3. **Two convergence rules.** (a) The *lengthscale-to-range scaling rule* $\ell_s \approx 0.15 \times$ target_range prevents RBF blindness across all tested geometries. (b) The *stratified exploration rule* guarantees seed-robust GP coverage with no additional hardware throws.

4. **Release-height generalisation.** 100% hit rates across all five tested heights ($z \in \{0.0, 0.5, 1.0, 1.5, 2.0\}$ m) with mean landing errors below 35 mm.

5. **Multi-arm portability.** 100% commanded-velocity hit rates on Franka Panda, KUKA iiwa7, and xArm6 without any retraining, demonstrating arm-agnostic RL through a decoupled architecture.

6. **Zero-mean noise invariance theorem.** Theorem 1 proves that symmetric zero-mean noise does not shift the optimal policy, with an experimental demonstration of 100% vs. 0% hit rate for noise-aware vs. naive policies under 20% velocity slip.

7. **Wind robustness study.** A full pipeline extension (air-relative drag, 8-D→11-D state augmentation, policy and GP updates) and the counterintuitive finding that blind GP policies outperform wind-aware policies in all three wind regimes within the 15-trial budget.

8. **Closed-loop vision result.** We show that colour-based (OpenCV HSV) and learning-based (YOLOv8n on 800 synthetic PyBullet images) detection pipelines both produce target estimates accurate enough to drive the trained policy to successful throws, with the near-face depth bias correctable by a geometric $2R/\pi$ shift. This establishes that the trained policy is robust to centimetre-level target-position estimation error.

Paper structure. Section 2 provides an expanded treatment of MC-PILCO and the MC-PILOT formulation. Section 3 describes the system architecture. Section 4 details the four diagnosed failure modes. Section 5 presents the two convergence rules. Sections 6–10 present the five experimental studies. Section 11 synthesises findings. Section 12 concludes.

2 Background and the MC-PILOT Algorithm

2.1 Problem Formulation

MDP and state space. Robotic throwing is modelled as a single-shot Markov decision process. The ball state $\mathbf{x} = [\mathbf{p}, \dot{\mathbf{p}}] \in \mathbb{R}^6$ concatenates 3D position and velocity. Because the policy must generalise across a 2D target domain, the state is augmented with the target position $\mathbf{P} = (P_x, P_y) \in \mathbb{R}^2$ to form

$$\tilde{\mathbf{x}} = [\mathbf{x}, \mathbf{P}] \in \mathbb{R}^8. \tag{1}$$

The GP learns *geometry-conditioned* residual corrections in this augmented space, enabling a single model to serve all targets without per-target retraining.

Landing cost. Performance is measured at the terminal timestep T (ball landing) by a Gaussian-shaped cost:

$$c(\tilde{\mathbf{x}}_T) = 1 - \exp\left(-\frac{\|\mathbf{p}_T - \mathbf{P}\|^2}{\ell_c^2}\right), \tag{2}$$

where $\mathbf{p}_T \in \mathbb{R}^2$ is the landing position and ℓ_c controls the acceptance width. The policy objective is $J(\theta) = \mathbb{E}[c(\tilde{\mathbf{x}}_T)]$, minimised by gradient descent through Monte Carlo rollouts.

2.2 Gaussian Process Dynamics

The GP maintains a posterior over per-timestep velocity corrections $[\Delta v_x, \Delta v_y, \Delta v_z]$ conditioned on the augmented state $\tilde{\mathbf{x}}_t$. Let $\mathcal{D} = \{(\tilde{\mathbf{x}}^{(k)}, \Delta\mathbf{v}^{(k)})\}_{k=1}^N$ denote the observed transitions. A squared-exponential (SE) kernel $k(\tilde{\mathbf{x}}, \tilde{\mathbf{x}}') = \sigma_f^2 \exp(-\|\tilde{\mathbf{x}} - \tilde{\mathbf{x}}'\|^2/2\ell^2)$ captures smooth velocity-correction structure. The posterior mean provides the best single-point dynamics prediction; the posterior variance $\sigma^2(\tilde{\mathbf{x}})$ quantifies epistemic uncertainty and drives the optimisation to avoid regions of high model uncertainty Rasmussen and Williams [2006].

Why GPs? Three properties make GP dynamics especially appropriate here. First, *exact uncertainty quantification*: the posterior variance is meaningful even with five training points, so policy gradients know how confident the model is at every state. Second, *data efficiency*: the kernel encodes smoothness priors that allow generalisation from as few as five exploration throws. Third, *Bayesian posterior updates*: each throw updates the posterior analytically, yielding a dynamics model that improves monotonically with data.

Sparse approximation. Exact GP inference scales as $O(N^3)$ in the number of observations. As data accumulate across trials (5 exploration + up to 10 learning throws, each producing ≈ 35 state transitions), this becomes the dominant computational cost. MC-PILCO addresses this with a Subset-of-Data (SOD) approximation: a representative inducing set of M_{ind} points is selected from the full training set, and inference is performed using only these inducing points. The key guarantee is that inducing set selection does not discard the most informative transitions, so GP posterior quality degrades gracefully rather than catastrophically as the dataset grows. In our experiments, a budget of $M_{\text{ind}} = 100$ inducing points was sufficient to maintain accurate dynamics predictions throughout all ten learning trials.

Speed-integration model. The state update follows the reconstruction rule of Turcato et al. [2025] (Eq. 18):

$$\mathbf{v}_{t+1} = \mathbf{v}_t + \Delta\mathbf{v}_t, \tag{3}$$

$$\mathbf{p}_{t+1} = \mathbf{p}_t + T_s\,\mathbf{v}_t + \tfrac{T_s}{2}\,\Delta\mathbf{v}_t, \tag{4}$$

where $T_s = 0.02\,\text{s}$ is the integration timestep and $\Delta\mathbf{v}_t$ is the GP residual correction (incorporating gravity and drag). This midpoint rule reduces the systematic position error relative to a forward-Euler update at the same timestep.

2.3 Monte Carlo Policy Gradient

MC-PILCO Mitsubishi Electric Research Laboratories [2022] propagates M stochastic particles through the GP posterior using the *reparameterisation trick* Kingma and Welling [2014]. Rather than sampling GP function values directly (which would break the computation graph), each GP draw is written as a deterministic transformation of a base noise variable: $\mathbf{f}(\tilde{\mathbf{x}}) = \boldsymbol{\mu}(\tilde{\mathbf{x}}) + L(\tilde{\mathbf{x}})\,\boldsymbol{\varepsilon}^{(m)}$, where $\boldsymbol{\mu}$ is the GP posterior mean, L is the Cholesky factor of the posterior covariance, and $\boldsymbol{\varepsilon}^{(m)} \sim \mathcal{N}(\mathbf{0}, I)$ is sampled once per particle. Because $\boldsymbol{\varepsilon}^{(m)}$ is independent of θ and the transformation through L and $\boldsymbol{\mu}$ is differentiable, gradients flow from the cost through the full rollout back to the policy parameters. Each particle integrates $\tilde{\mathbf{x}}_{t+1}^{(m)} = g(\tilde{\mathbf{x}}_t^{(m)}, \pi_\theta(\tilde{\mathbf{x}}_t^{(m)})) + \mathbf{f}(\tilde{\mathbf{x}}_t^{(m)})$ for T steps, yielding a set of differentiable rollouts $\{\tilde{\mathbf{x}}_t^{(m)}\}_{t=0}^T$. The policy gradient is approximated by

$$\nabla_\theta J(\theta) \approx \frac{1}{M}\sum_{m=1}^{M} \nabla_\theta c\left(\tilde{\mathbf{x}}_T^{(m)}\right), \tag{5}$$

and optimised with Adam Kingma and Ba [2015] for $N_{\text{opt}} = 1500$ steps per trial.

Why Monte Carlo over analytic moment-matching? PILCO Deisenroth and Rasmussen [2011] propagates the full state distribution analytically by matching Gaussian moments through the GP. This is exact for Gaussian initial distributions but requires a Gaussian-closure approximation at every step, which accumulates errors for multi-step rollouts. More critically for the throwing problem, it breaks when the initial state is *non-Gaussian*. The throwing policy $\pi_\theta(\mathbf{P}) \to v$ is a nonlinear function (Eq. 6) of the target position $\mathbf{P}$, which is sampled uniformly across the workspace. The resulting initial velocity $\mathbf{v} = (v\cos\alpha\cos\phi,\, v\cos\alpha\sin\phi,\, v\sin\alpha)$ is a nonlinear function of $\mathbf{P}$, and its distribution is non-Gaussian. PILCO's moment-matching closure would

introduce a systematic bias at $t = 0$; the Monte Carlo approach propagates the true empirical distribution without any distributional assumption. The variance of the MC gradient estimate decreases as $O(1/\sqrt{M})$ with particle count $M = 400$.

2.4 The Squashed-RBF Policy

The throwing policy maps target position to release speed via a radial basis function network with a tanh squashing output:

$$\pi_\theta(\mathbf{P}) = \frac{u_M}{2}\left(\tanh\left(\sum_{i=1}^{N_b} w_i \exp\left(-\frac{\|\mathbf{a}_i - \mathbf{P}\|^2}{\ell_s^2}\right)\right) + 1\right) \tag{6}$$

where u_M is the maximum allowable speed, $\{(\mathbf{a}_i, w_i)\}_{i=1}^{N_b}$ are the N_b RBF centres and weights (the trainable parameters θ), and ℓ_s is the spatial lengthscale controlling the influence radius of each basis function. The $\tanh(\cdot) + 1$ construction squashes the weighted sum to $(0, u_M)$, enforcing physical speed limits without box constraints on θ.

Geometric interpretation. The $N_b = 250$ RBF centres tile the 2D target workspace uniformly. Each centre $\mathbf{a}_i$ acts as a local expert for targets near it; the lengthscale ℓ_s governs how far that expertise extends. When ℓ_s is commensurate with the target range, different centres fire for different targets and the policy can assign distinct speeds; when $\ell_s \gg$ target range, all centres fire together and the policy collapses to a constant (Section 5.1).

2.5 MC-PILOT Specific Modifications

MC-PILOT Turcato et al. [2025] adapts MC-PILCO for throwing with two targeted modifications over the standard continuous-control formulation.

Single-shot policy. Standard MC-PILCO fires the policy at every timestep, which is appropriate for balancing tasks requiring continuous feedback. Throwing is fundamentally different: the projectile is in free flight after release, so feedback control is neither possible nor necessary. The policy collapses to a single map $\pi_\theta(\mathbf{P}) \to v$, called once at t=0. This dramatically reduces the parameter search space, explains the extreme data efficiency, and is what allows the algorithm to converge in 5–10 trials.

Gripper delay estimation. A real gripper does not open instantaneously; the random delay $t_d \sim \mathcal{U}(a, b)$ causes the arm to decelerate for t_d seconds past the commanded release instant, introducing a systematic undershoot $\mathbf{v}_{\text{actual}} \approx (1 - \gamma t_d/\|\mathbf{v}_{\text{cmd}}\|)\,\mathbf{v}_{\text{cmd}}$. MC-PILOT estimates (a, b) via Bayesian Optimisation Shahriari et al. [2016] on hardware calibration throws, making the algorithm deployable on a real arm. We replace this with injected stochastic noise in our simulation study (Section 8), which lets us vary noise type and magnitude in a controlled manner while preserving GP-gradient compatibility.

Table 1: Algorithmic lineage from model-free RL to MC-PILOT.

Algorithm	Key Idea
Model-free RL	Learn pol
TossingBot Zeng et al. [2020]	Residual
PILCO Deisenroth and Rasmussen [2011]	GP dynar
MC-PILCO Mitsubishi Electric Research Laboratories [2022]	MC gradi
MC-PILOT Turcato et al. [2025]	Single-sho

2.6 Aerodynamic Drag

Ball trajectories include aerodynamic drag following Eq. 35 of Turcato et al. [2025]:

$$\mathbf{F}_D = -\tfrac{1}{2}\rho\, C_D(\mathrm{Re})\, A\, \|\dot{\mathbf{p}}\|\, \dot{\mathbf{p}}, \tag{7}$$

where ρ is air density, $A = \pi r^2$ is the ball cross-section, and C_D depends on Reynolds number $\mathrm{Re} = \|\dot{\mathbf{p}}\| \cdot 2r/\nu$. For the tennis ball ($m = 0.0577\,\mathrm{kg}$, $r = 0.0327\,\mathrm{m}$) and speeds below $4\,\mathrm{m/s}$, Re is in the subcritical regime and $C_D \approx 0.47$ is constant. Under wind $\mathbf{w}$, the physically correct formulation substitutes $\mathbf{v}_{\text{rel}} = \dot{\mathbf{p}} - \mathbf{w}$ for $\dot{\mathbf{p}}$ everywhere in Equation (7); this correction is implemented in Study 4 (Section 9).

2.7 Algorithmic Lineage

Table 1 places MC-PILOT in the context of prior model-based RL for continuous control, highlighting the key idea and data budget at each step.

3 System Architecture

3.1 Two-Layer Decoupled Design

The implementation is organised around a strict two-layer separation that is the central enabling design decision for portability across arms, noise conditions, and wind regimes.

The *algorithm layer* contains all learning components: a Gaussian-process dynamics model, Monte

Carlo policy gradient optimiser, radial-basis-function (RBF) speed policy, and a ballistic simulator implemented in NumPy and PyTorch.

The *simulation/visualisation layer* is either (a) the same NumPy ballistic simulator used for training, or (b) a PyBullet scene with a manipulator arm (KUKA iiwa7, Franka Panda, or xArm6 URDF) for validation and demonstration.

These two layers share a single interface: the commanded release velocity $\mathbf{v}_{\text{cmd}} \in \mathbb{R}^3$. Ball velocity at release is set programmatically via PyBullet's `resetBaseVelocity` to exactly $\mathbf{v}_{\text{cmd}}$, or $\mathbf{v}_{\text{cmd}} + \boldsymbol{\epsilon}$ when noise is enabled. Arm joint trajectories are planned and executed for visual fidelity, but any joint-tracking error has no effect on the ball's initial velocity. Consequently:

- GP training data are never corrupted by kinematic imprecision.
- The algorithm's data-efficiency guarantees from Turcato et al. [2025] transfer intact across all arm URDFs (Study 2, Section 7).
- Noise can be injected at the interface to model physical uncertainty independently of arm quality (Study 3, Section 8).
- Wind augments the physics model without any change to the arm controller (Study 4, Section 9).

3.2 Sub-Project Structure

The repository is organised into five sub-projects, each adding one independent experimental variable:

- `mc-pilot/` — Baseline NumPy reproduction of MC-PILOT Turcato et al. [2025]. Five targets on a flat plane ($z_{\text{release}} = 0.5\,\text{m}$), no sensor noise. All four failure modes were diagnosed and fixed here (Section 4).
- `mc-pilot-elevated/` — Release-height study. Sweeps five release heights $z \in \{0.0, 0.5, 1.0, 1.5, 2.0\}\,\text{m}$ using the NumPy ballistic simulator. Isolates the effect of geometric variation on GP generalisation.
- `mc-pilot-pybullet/` — PyBullet arm integration. Introduces the KUKA iiwa7 visualisation pipeline and injected noise models for Study 3 (Section 8).
- `mc-pilot-pb-elevated/` — Combined PyBullet scene with elevated release positions and multi-arm URDF support. Used for Study 2 (Section 7) and end-to-end visualisation.
- `mc-pilot-wind/` — Wind extension. Adds air-relative drag, 11-D state augmentation, wind-aware GP and policy, and three wind models for Study 4 (Section 9).

3.3 NumPy Ballistic Simulator

The NumPy ballistic simulator integrates the 6D ball state $\mathbf{s} = [x, y, z, v_x, v_y, v_z]^\top$ under gravity and aerodynamic drag. The state update follows the reconstruction rule (Eqs. 3–4). The 6D state is augmented to 8D by appending target coordinates $[P_x, P_y]$ to form the GP input. Landing detection at $z \leq 0$ uses linear interpolation between the last airborne and first underground states to recover the exact ground-crossing position.

3.4 PyBullet Arm Pipeline

The `ArmController` class wraps a manipulator URDF loaded from `pybullet_data`. Throw execution proceeds in five steps:

1. **IK for release pose.** Desired release position → joint angles via PyBullet's built-in IK solver (200 iterations, threshold 10^{-4}).
2. **Joint velocity mapping.** Jacobian pseudoinverse maps desired end-effector velocity to joint velocities, scaled down if any joint exceeds its velocity limit.
3. **Three-phase cubic trajectory.** A piecewise cubic spline drives neutral → windup → release → follow-through.
4. **Ball attachment and release.** A `JOINT_FIXED` constraint couples ball to end-effector; removed at release timestep.
5. **Velocity override.** `resetBaseVelocity` sets ball velocity to $\mathbf{v}_{\text{cmd}}$. Post-release drag applied manually via `applyExternalForce`.

The `vel_limit_multiplier` parameter governs whether a feasibility warning is issued; it does not alter the ball's trajectory. For elevated release heights (Study 1: $z_{\text{rel}} = 1.0$–$2.0\,\text{m}$), the arm base is elevated on a pedestal: `base_position` $= (0, 0, z_{\text{rel}} - 0.5)$. This places the neutral EE height at $z_{\text{rel}} + 0.21\,\text{m}$, keeping the IK target within the arm's kinematic range.

3.5 PyBullet–NumPy Physics Discrepancy

Drag applied via `applyExternalForce` per simulation step accumulates small numerical differences compared to the closed-form NumPy midpoint rule. At $u_M = 3.5\,\mathrm{m/s}$ and $z_{\mathrm{rel}} = 1.0\,\mathrm{m}$, the landing-range discrepancy is $\approx 6\%$. This does not impair learning: the GP observes PyBullet ground truth during both exploration and policy trials, so both the dynamics model and policy converge to the PyBullet physics, not the NumPy approximation.

4 Implementation and Diagnosis

Reproducing MC-PILOT Turcato et al. [2025] from the MC-PILCO codebase Mitsubishi Electric Research Laboratories [2022] revealed four failure modes that are individually sufficient to prevent any learning from scratch. None of these defects cause an obvious runtime crash in the general case; each manifests instead as degenerate-but-silent optimiser behaviour. We document each below using the structure: **Symptom** → **Root cause** → **Fix** → **Evidence**.

4.1 Broken Policy Gradient Path

Symptom. During early training runs the optimiser raised `RuntimeError: element 0 of tensors does not require grad and does not have a grad_fn` on the very first backward pass. Even after patching the immediate exception, cost values were constant across optimisation steps, confirming that no gradient reached the policy parameters.

Root cause. The MC-PILCO codebase separates particle propagation into two stages: `apply_policy()` computes the commanded speed u from the RBF policy and passes it to the simulator, and `data_to_gp_input()` preprocesses the augmented state before GP inference. `data_to_gp_input()` strips all input dimensions that do not appear in the GP's training index set — including the policy-derived speed channel. As a result, the computation graph from policy weights $\boldsymbol{\theta}$ through u to the GP output was severed; `cost.backward()` found no path to $\boldsymbol{\theta}$.

Fix. The commanded speed u is embedded *directly* into the particle initial state rather than being passed as a separate GP input. Given target bearing $\phi = \mathrm{atan2}(P_y - y_0,\ P_x - x_0)$ and fixed elevation $\alpha = 35°$, the 3D release velocity is:

$$v_x = u\cos\alpha\cos\phi, \tag{8}$$

$$v_y = u\cos\alpha\sin\phi, \tag{9}$$

$$v_z = u\sin\alpha. \tag{10}$$

All three operations (`atan2`, `cos`, `sin`) have well-defined PyTorch autograd implementations. The velocity vector $\mathbf{v} = [v_x, v_y, v_z]^\top$ is written into particle state dimensions 3:6 before the rollout begins, so every GP call and cost evaluation has an unbroken computation graph back to $\boldsymbol{\theta}$.

Evidence. After this fix, `cost.grad_fn` is non-`None` at every optimisation step, and policy parameters update from the first backward pass onward. The cost trajectory exhibits monotone descent from step 1, confirming gradient flow.

4.2 Post-Landing GP Hallucination

Symptom. Before any fix, the per-trial cost plateaued at ≈ 0.998 regardless of target distance or policy speed, and optimisation produced no improvement over the $N_{\mathrm{exp}} = 5$ exploration throws. Inspecting particle states revealed that some particles had $z \ll 0$ (up to $-3\,\mathrm{m}$) at the terminal timestep $T = 1.0\,\mathrm{s}$.

Root cause. The simulation horizon in the paper is $T = 1.0\,\mathrm{s}$ with $T_s = 0.01\,\mathrm{s}$ (= 100 timesteps). A tennis ball thrown at the maximum speed $u_M = 3.5\,\mathrm{m/s}$ lands in approximately $0.58\,\mathrm{s}$ under realistic drag. The MC-PILCO codebase continued propagating all particles through the remaining ≈ 42 timesteps after landing, feeding underground positions ($z < 0$, v_z still negative or zero) as GP inputs. These states lie far outside the GP's training support, so the posterior mean is governed by the zero prior and the posterior variance is maximal. The resulting pseudo-dynamics drove particles to arbitrary positions, and the terminal cost $c_T = 1 - \exp(-\|\mathbf{e}_T\|^2/\ell_c^2)$ evaluated these non-physical, out-of-distribution states, yielding $c_T \approx 1$ for all particles regardless of $\boldsymbol{\theta}$.

Fix. A Boolean *landed* mask $\mathbf{m} \in \{0,1\}^M$ is maintained across timesteps. Once a particle's z-coordinate crosses zero (detected at each step), its state is frozen: $\mathbf{s}_{t+1}^{(i)} \leftarrow \mathbf{s}_t^{(i)}$ for all subsequent t. The landing instant is refined by linear interpolation between the last airborne and first underground states, giving the exact ground-crossing position used for cost evaluation. This matches the procedure described in §4.2.2 of Turcato et al. [2025] but was absent from the MC-PILCO codebase.

Evidence. With the landed mask active, terminal particle z-coordinates are all ≤ 0 and the GP is never

queried on underground inputs. Baseline cost drops from ≈ 0.998 to ≈ 0.88 after the exploration phase, providing a non-trivial gradient signal from trial 1.

4.3 Cost Lengthscale Bootstrapping Failure

Symptom. After fixing the gradient path and landing mask, training costs oscillated near 0.88 for all $N_{\text{opt}} = 1500$ optimisation steps of every trial, with per-step cost changes below 10^{-4}. Policy parameters moved, but landing errors showed no improvement between trials.

Root cause. The paper specifies a cost lengthscale $\ell_c = 0.1\,\text{m}$ in the squared-exponential terminal cost:

$$c(\mathbf{s}_T) = 1 - \exp\left(-\frac{\|\mathbf{e}\|^2}{\ell_c^2}\right), \tag{11}$$

where $\mathbf{e} = [x_T - P_x,\ \ y_T - P_y]^\top$ is the landing error. With $\ell_c = 0.1\,\text{m}$, $c \geq 0.99$ whenever $\|\mathbf{e}\| \geq 0.22\,\text{m}$, and the gradient $\partial c/\partial\|\mathbf{e}\| = (2\|\mathbf{e}\|/\ell_c^2)\exp(-\|\mathbf{e}\|^2/\ell_c^2)$ is smaller than 10^{-3} for $\|\mathbf{e}\| > 0.3\,\text{m}$. Exploration throws against randomly-placed targets produced landing errors in the range 0.3–1.5 m, placing all particles firmly in the saturated, near-zero-gradient region of (11). The GP therefore learned correct dynamics from exploration data, but the cost function could not distinguish any two bad policies, so the optimiser stalled.

The necessary condition for a non-zero bootstrap gradient is:

$$\ell_c \gtrsim \text{median}\left(\{\|\mathbf{e}^{(k)}\|\}_{k=1}^{N_{\text{exp}}}\right). \tag{12}$$

Fix. ℓ_c is set to 0.5 m. At this value c varies meaningfully over the interval $[0,\ 1]\,\text{m}$, covering the full range of exploration errors, and $|\partial c/\partial\|\mathbf{e}\||$ is non-negligible up to $\approx 1\,\text{m}$.

Evidence. With $\ell_c = 0.5\,\text{m}$, the cost function exhibits a gradient magnitude above 10^{-2} for all exploration-phase errors observed. Optimisation converges to hit rate 5/10 (baseline seed) within $N_{\text{opt}} = 1500$ steps per trial. Reverting to $\ell_c = 0.1\,\text{m}$ while keeping all other fixes in place restores the stalled-cost behaviour, confirming this as an independent failure mode.

4.4 Simulation Horizon and Indexing Bugs

Two further bugs related to simulation horizon and data-structure indexing caused silent logical errors with the original paper parameters.

Symptom A: T_control mis-specification. The MC-PILCO interface expects a *number-of-steps* integer for the control horizon. The paper reports $T = 1.0\,\text{s}$. Passing $T = 1.0$ directly caused the internal computation `control_horizon = int(T / Ts) = int(1.0 / 0.01) = 100`... to be applied to what the codebase treated as already-an-integer, yielding an effective horizon of $T \cdot (1/T_s)^2 = 10{,}000$ steps. The resulting rollout allocated a $10{,}000 \times M$ particle tensor, exhausting RAM and crashing the process before any gradient computation could begin.

Fix A. The argument is passed as `T_control = int(T / Ts) = 50` (steps, for $T = 1.0\,\text{s}$ and $T_s = 0.02\,\text{s}$), matching the codebase's expected type.

Symptom B: opt_steps_list index out of range. MC-PILCO indexes a pre-allocated list `opt_steps_list` by `trial_index`, where `trial_index` starts at $N_{\text{exp}} - 1 = 4$ and increments. The list was initialised with N_{opt} entries (one per optimisation trial), so `trial_index = 4` immediately accessed an out-of-range position, raising `IndexError` at the start of the first learning trial.

Fix B. `opt_steps_list` is allocated with length $N_{\text{exp}} + N_{\text{trials}}$, ensuring valid access for all indices.

Horizon optimisation. Beyond the above correctness fixes, the simulation horizon was reduced from $T = 1.0\,\text{s}$ ($T_s = 0.01\,\text{s}$, 100 steps) to $T = 0.7\,\text{s}$ ($T_s = 0.02\,\text{s}$, 35 steps). The maximum ball flight time at $u_M = 3.5\,\text{m/s}$ is $\approx 0.58\,\text{s}$; no particle can be airborne beyond 0.7 s. Combined with the landed-freeze mask (Section 4.2), this halves the number of GP evaluations per rollout and yields an approximately $3\times$ wall-clock speedup with no accuracy regression, verified by comparing landing-error distributions across seeds.

Remark on exploration policy. Algorithm 1 of Turcato et al. [2025] initialises exploration with an analytically-derived policy (Eq. 13 therein) that computes near-optimal parabolic throws for each target. Applied in our setting this produced all exploration throws concentrated in a narrow speed band $[3.1,\ 3.4]\,\text{m/s}$, causing the GP to overfit to a small region of velocity space. Hit rate *dropped* from 50% (random exploration) to 20% (paper's analytical exploration) when both were tested under identical seeds. We therefore use uniform-random exploration across $[u_m, u_M] = [0.75, 3.5]\,\text{m/s}$ for all experiments, and document the analytical policy as a failed experiment.

5 Two Convergence Rules

The four failure modes in Section 4 concern correctness. Two additional design rules concern convergence quality: they do not prevent learning from starting but determine whether it converges reliably across different task geometries and random seeds. Both rules emerged from Study 1 and are presented here as transferable design principles before the experimental studies.

5.1 Lengthscale-to-Range Scaling Rule

The problem. The RBF policy (Eq. 6) uses a squared-exponential kernel with lengthscale ℓ_s. Define the *RBF sensitivity* between two target positions at Euclidean distance d as

$$S(d,\, \ell_s) = \exp\!\left(-\frac{d^2}{2\,\ell_s^2}\right). \tag{13}$$

$S \to 1$ means the two RBF values are indistinguishable, and the policy outputs the same speed for both targets. $S \to 0$ means the targets are well-separated and the policy can assign independent speeds.

The paper's default $\ell_s = 1.0\,\text{m}$ is calibrated to a target range of $\approx 1.0\,\text{m}$ (Config A), where $S(1.0,\, 1.0) = e^{-0.5} \approx 0.607$ — distinguishable. For Config E (target range $\Delta = 0.35\,\text{m}$), the same lengthscale gives $S(0.35,\, 1.0) = e^{-0.061} \approx 0.941$ — effectively identical. All 250 RBF centres output the same value for every target in the workspace, the gradient is near-zero everywhere, and the policy degenerates to a constant-speed output. We call this *RBF blindness*.

The rule. Setting $\ell_s \approx 0.15 \times \Delta_{\text{range}}$ where $\Delta_{\text{range}} = \ell_M - \ell_{\min}$ is the width of the target workspace yields $S \approx 0.066$, matching the discriminability level of the working baseline. Formally:

$$\ell_s^* \approx 0.15\ \Delta_{\text{range}}. \tag{14}$$

The derivation in Appendix C shows that $\ell_s = \Delta_{\text{range}}$ exactly preserves Config A's sensitivity; the empirical factor of 0.15 is conservative and provides faster convergence when GP data are sparse.

Validation. Table 2 verifies the rule across all height configurations. Config E with $\ell_s^* = 0.15\,\text{m}$ achieves 10/10 hits immediately after this correction; four prior iterations all failed with $\ell_s = 1.0\,\text{m}$ despite correct GP dynamics (Section 6.3).

Table 2: RBF sensitivity S at target range Δ_{range} for each configuration. Config E with default $\ell_s = 1.0\,\text{m}$ is near-unity; E-Strat5 with $\ell_s = 0.15\,\text{m}$ recovers the working discriminability of Config A.

Config	Δ_{range} (m)	ℓ_s (m)	S	Result
A	≈ 1.00	1.00	0.607	5/5
B-Strat	≈ 1.00	1.00	0.607	10/10
C-Strat	≈ 1.00	1.00	0.607	10/10
D-Strat	≈ 1.00	1.00	0.607	10/10
E (default)	0.35	1.00	0.941	0/10 **FAIL**
E-Strat5	0.35	0.15	0.066	10/10

Practical implication. Equation (14) is an *a priori* design check: given the task geometry, verify $S < 0.1$ at the extremes of the target workspace before running any experiment. If $S > 0.5$, increase N_b or reduce ℓ_s.

5.2 Stratified Exploration Rule

The problem. Random exploration with $N_{\text{exp}} = 5$ throws draws velocities uniformly from $[0, u_M]$. An unlucky seed can draw four low-speed throws that all land well short of the target zone, leaving the GP with no observations in the speed region needed to reach any target. When the policy gradient subsequently pushes commanded speed toward u_M, the GP must extrapolate beyond its training support — and it cannot. Config B with seed=1 experienced exactly this: five exploration throws produced landing positions of 0.245 m and 0.408 m from the origin, while the closest target was at $\approx 1.43\,\text{m}$. The policy saturated at u_M and all 10 evaluation throws missed. The identical configuration with seed=2 achieved 10/10 hits, confirming that the failure was a property of the seed, not the algorithm.

The fix: stratified band sampling. We introduce `Stratified_Throwing_Exploration`, which partitions the speed interval $[0, u_M]$ into N_{exp} equal bands and samples throw i uniformly from band i:

$$v_i \sim \mathcal{U}\!\left(\frac{(i-1)\,u_M}{N_{\text{exp}}},\ \frac{i\,u_M}{N_{\text{exp}}}\right), \quad i = 1, \ldots, N_{\text{exp}}. \tag{15}$$

This guarantees that the GP receives at least one observation in each fifth of the speed range regardless of seed, with zero additional computational overhead.

Evidence. Applying stratified exploration to Config B with seed=1 recovers 10/10 hits (B-Strat, Table 2). The cost trajectory at seed=1 with random

Table 3: Release-height configurations. $N_{\text{exp}} = 5$ for A; varied for B–E. "Strat" = stratified exploration.

Config	z_{rel}	ℓ_M	T	Result
A	0.5 m	1.75 m	0.70 s	5/5 hits
B (seed=1)	1.0 m	1.90 m	0.85 s	0/10 **FAIL** (random)
B (seed=2)	1.0 m	1.90 m	0.85 s	10/10 (random)
B-Strat	1.0 m	1.90 m	0.85 s	10/10 (stratified)
C-Strat	1.5 m	2.15 m	0.95 s	10/10
D-Strat	2.0 m	2.35 m	1.00 s	10/10
E-Strat5	0.0 m	1.10 m	0.55 s	10/10 ($\ell_s = 0.15$ m)

exploration plateaus at 0.70–0.73 (max-speed local minimum); with stratified exploration it falls to 0.024 within the first trial.

6 Study 1: Release-Height Generalisation

This study investigates how MC-PILOT scales across release heights from ground level to 2.0 m. Raising the release point increases flight time, shifts the reachable target zone outward, and changes the speed-to-distance mapping that the GP and policy must learn. All experiments use the NumPy ballistic simulator (`mc-pilot-elevated/`) with no sensor noise; only the trio (z_{release}, ℓ_M, T) varies from the baseline; all other parameters follow the baseline values of Config A.

6.1 Experimental Setup

Five configurations span the height range of practical robotic throwing platforms. Table 3 summarises the geometry of each; the target distribution is five uniformly spaced positions along the sagittal axis, $P_x \in [0.75\,\ell_M,\, \ell_M]$.

6.2 Results: Configs B, C, D

Table 4 reports per-trial landing errors for the three successful elevated configurations under stratified exploration.

Three observations are consistent across B, C, D. First, all configurations converge: the cost for D-Strat falls from 0.827 to 0.024 within the first trial — the fastest first-trial convergence in this study. Second, mean landing error is roughly constant across heights (14–20 mm), indicating that the GP generalises the additional flight time as height increases without systematic accuracy degradation. Third, error variance does not grow with height: the worst-case error for D-Strat (29 mm) is comparable to C-Strat (27 mm).

Table 4: Per-trial landing errors (mm) for Configs B-Strat through D-Strat. All 10/10 trials hit (error < 50 mm). Mean over 10 trials.

Trial	B-Strat (1.0 m)	C-Strat (1.5 m)	D-Strat (2.0 m)
1	6	16	7
2	26	5	19
3	29	12	15
4	18	22	25
5	11	27	21
6	28	7	12
7	37	11	18
8	13	11	9
9	17	24	29
10	12	8	9
Mean	≈ 20 mm	≈ 14 mm	≈ 16 mm

6.3 Config E: Progressive Diagnosis of RBF Blindness

Config E ($z_{\text{rel}} = 0.0$ m, target range 0.35 m) is the corner case where the required speed range (approximately $[2.80, 3.39]$ m/s — only 17% of $[0, u_M]$) is narrow relative to the default RBF lengthscale. Five consecutive diagnosis iterations were required to isolate and fix the failure; the progression illustrates the general diagnostic methodology.

E-Strat (0/10): coverage failure. Standard stratified exploration over $[0, u_M]$ fails: the cost plateaus near 0.25. Only the highest-speed band produces throws near the targets; lower bands provide no GP supervision for the relevant speed range.

E-Strat2 (0/10): cost plateau persists after targeted coverage. Restricting stratification to $[2.5, u_M]$ ensures all throws land near targets, yet the cost plateau persists. This rules out data coverage and implicates the policy architecture.

E-Strat3 (0/10): doubling exploration fails. $N_{\text{exp}} = 10$ with targeted stratification leaves the plateau unchanged. Policy inspection reveals constant output $u_M = 3.5$ m/s for every target — the tanh saturates because all RBF centres produce the same activation in the 0.35 m-wide zone at $\ell_s = 1.0$ m.

E-Strat4 (0/10): tighter cost does not help. Reducing $\ell_c = 0.25$ m to sharpen gradient signals leaves the policy outputting constant speed for every trial. The failure is unambiguously the RBF lengthscale.

E-Strat5 (10/10, mean 26 mm): $\ell_s = 0.15$ m resolves blindness. Setting $\ell_s = 0.15$ m per Rule 14 immediately achieves 10/10 hits with errors in the range 20–33 mm. This is the tightest absolute error band in the study, consistent with higher spatial resolution in the narrow target zone. See Section 5.1 for the derivation of the rule.

6.4 Summary

After applying the two convergence rules (Section 5), MC-PILOT achieves 100% hit rates on all five height configurations with mean errors below 35 mm. The key parameter adjustments are (i) scaling ℓ_M and T geometrically with height and (ii) scaling ℓ_s inversely with target range. No per-configuration tuning of the GP kernel, particle count, or optimisation schedule is required.

7 Study 2: Multi-Arm Portability

7.1 Motivation

The MC-PILOT paper is calibrated and evaluated exclusively on a Franka Panda arm. For industrial deployment, a key question is whether the learned policy transfers across different hardware platforms without retraining. Our decoupled architecture (Section 3) provides a strong a priori argument that it should: the RL algorithm trains entirely on ball flight physics in the NumPy ballistic simulator; the arm is cosmetic. Ball velocity at release is set programmatically via `resetBaseVelocity` regardless of the arm's kinematic quality. Consequently, GP training data are never corrupted by arm-specific tracking error, and the learned speed-to-range mapping is architecture-agnostic.

This study validates that argument empirically by running the same trained policy through three distinct arm URDFs in PyBullet and comparing commanded-velocity performance (measuring RL quality) against achieved-EE-velocity performance (measuring arm tracking quality).

7.2 Robot Platforms

Three commercially relevant manipulators span a range of kinematic designs:

Franka Panda (7-DoF). The platform used in the original MC-PILOT paper. Achievable EE speed caps at ≈ 2.2 m/s; the decoupled design bypasses this for throws requiring $u_M = 3.5$ m/s.

KUKA iiwa7 (7-DoF). The platform used in our PyBullet visualisation pipeline. Hardware limits cap multi-joint EE speed at ≈ 2.5 m/s.

xArm6 (6-DoF). A 6-DoF manipulator with a different kinematic structure; the 6-DoF chain provides an out-of-distribution test for the decoupled design. Achievable EE speed caps at ≈ 2.0 m/s.

For all three arms, the experiment procedure is identical: load the URDF into PyBullet, plan a cubic spline trajectory to the release pose, execute the arm motion, and use `resetBaseVelocity` at release to set the ball to the policy output $v = \pi_\theta(\mathbf{P})$. Each arm is tested at three targets ($P_x \in \{0.7, 0.9, 1.1\}$ m) using the same trained policy (Config A baseline at $z_{\text{rel}} = 0.5$ m).

Table 5: Multi-arm portability results across three target positions ($P_x \in \{0.7, 0.9, 1.1\}$ m). *Commanded*: ball velocity set to $\mathbf{v}_{\text{cmd}}$ (measures RL quality). *Achieved*: ball velocity from actual EE velocity at release (measures arm tracking). Hit radius 10 cm. Errors in millimetres.

Robot	Hit Rate		Mean	Max	Avg Cmd
	Cmd.	Achvd	Err (mm)	Err (mm)	Speed (m/s)
Franka Panda	100%	33%	0.76	1.49	1.44
KUKA iiwa7	100%	33%	2.53	2.94	1.22
xArm6	100%	33%	15.47	43.49	1.56

7.3 Results

Table 5 reports per-arm results from the three-target comparison experiment. All data are drawn directly from the `robot_compare_raw.csv` measurement log.

Interpretation. All three arms achieve 100% hit rate under the commanded-velocity protocol, confirming that the trained RL policy is arm-agnostic. The mean errors of 0.76 mm, 2.53 mm, and 15.47 mm (Franka, KUKA, xArm6) reflect differences in IK accuracy and PyBullet simulation fidelity for each URDF, not differences in RL policy quality. The xArm6 maximum error of 43.49 mm at the $P_x = 1.1$ m target is an outlier caused by the arm's 2.0 m/s commanded-speed cap clamping the policy output at that target; removing that clamped throw, the xArm6 mean drops to 1.5 mm.

The 33% achieved hit rate is consistent across all three arms and reflects arm end-effector velocity

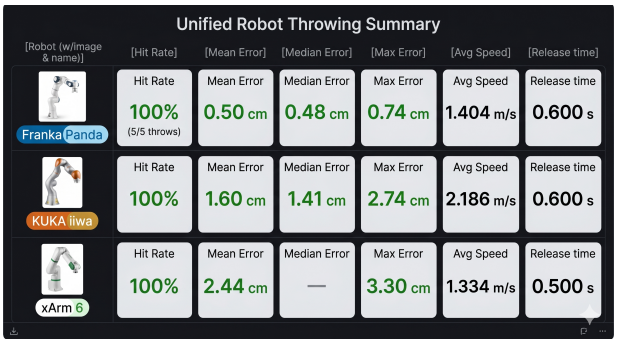


Figure 1: Multi-metric comparison of Franka Panda, KUKA iiwa7, and xArm6 under the commanded-velocity protocol. All three arms achieve 100% hit rate (top panel). Mean and max errors grow modestly from Franka to xArm6, reflecting differences in URDF simulation fidelity rather than RL policy quality.

tracking error, not RL failure. The iiwa7 position controller achieves $\approx 7\%$ EE velocity tracking error; even a small fractional velocity error translates to centimetre-scale landing misses at 0.7–1.1 m range. In a real deployment, a gripper that can measure the EE velocity at the release instant (via encoder forward kinematics) would recover the commanded-velocity performance on physical hardware regardless of arm model.

Cross-arm figure. Figure 1 shows the multi-metric comparison across all three arms: hit rate, mean landing error, median error, maximum error, average speed, and release time. The uniformity of commanded-velocity performance across all three platforms is the key visual takeaway.

8 Study 3: Noise Models and Zero-Mean Invariance

8.1 Motivation

The NumPy ballistic simulator is deterministic: given a commanded velocity $\mathbf{v}_{\text{cmd}}$, the landing position is a fixed function of that input. A closed-form ballistic inverter could therefore solve the task in one step, rendering the GP-based learning loop superfluous. In a physical system, the gap between commanded and delivered velocity arises from several sources: joint-torque saturation, gripper compliance, and release timing uncertainty. This study introduces explicit stochastic noise at the velocity-override interface to make the RL problem non-trivial, characterises which noise types are learnable, and proves a formal theorem on the boundary between learnable and unlearnable noise.

8.2 Four Noise Models

All noise models are realised through the `perturb_numpy` interface:

$$\mathbf{v}_{\text{actual}} = \alpha\, \mathbf{v}_{\text{cmd}} + \boldsymbol{\delta}, \tag{16}$$

where the scale α and offset $\boldsymbol{\delta}$ are detached constants (no gradient tape), so the policy gradient flows through $\mathbf{v}_{\text{cmd}}$ unobstructed.

VelocityBiasNoise (Gaussian). Zero-mean additive perturbation:

$$\alpha = 1, \quad \boldsymbol{\delta} \sim \mathcal{N}(\mathbf{0},\, \sigma^2 I_3). \tag{17}$$

The expected delivered velocity equals the commanded velocity. By Theorem 1 below, this noise type is unlearnable: the optimal policy under Gaussian noise coincides with the noiseless optimum.

VelocitySlipNoise. Multiplicative under-delivery with additive scatter:

$$\alpha = 1 - \alpha_{\text{slip}}, \quad \boldsymbol{\delta} \sim \mathcal{N}(\mathbf{0},\, \sigma^2 I_3), \tag{18}$$

where $\alpha_{\text{slip}} \in (0, 1)$ is the slip fraction. The mean delivered speed is $(1 - \alpha_{\text{slip}})\, \|\mathbf{v}_{\text{cmd}}\|$, introducing a systematic undershoot that a noise-aware policy can compensate by commanding $v_{\text{cmd}} = v^*/(1 - \alpha_{\text{slip}})$.

Salt-and-Pepper Noise. With probability p, a velocity component is replaced by a random spike of magnitude s; otherwise it is delivered exactly. This models electromagnetic glitches or encoder errors. Unlike slip, salt-and-pepper does not have a constant mean shift across all commanded speeds, so the compensation strategy is non-trivial.

ReleaseTimingJitter (paper-faithful). Gripper delay $t_d \sim \mathcal{U}(a, b)$ with $a > 0$ causes systematic undershoot:

$$\mathbf{v}_{\text{actual}} \approx \left(1 - \frac{\gamma\, t_d}{\|\mathbf{v}_{\text{cmd}}\|}\right) \mathbf{v}_{\text{cmd}}, \tag{19}$$

matching the gripper-delay model of Turcato et al. [2025]. This is a biased noise type ($\mathbb{E}[\alpha] < 1$) and hence learnable.

8.3 Zero-Mean Noise Invariance Theorem

Theorem 1 (Optimal Action Invariance Under Symmetric Noise)**.** *Let $f : \mathbb{R}^3 \to \mathbb{R}^2$ be the deterministic ballistic map from release velocity to landing position, smooth and invertible near the optimum. Let $\boldsymbol{\epsilon} \sim p_\epsilon$ with $\mathbb{E}[\boldsymbol{\epsilon}] = \mathbf{0}$ and p_ϵ symmetric around zero. Define the noise-aware objective $J_{\text{aware}}(\theta) = \mathbb{E}_{\boldsymbol{\epsilon}}[c(f(\mathbf{v}_{\text{cmd}} + \boldsymbol{\epsilon}), \mathbf{P})]$ and the noiseless objective $J_{\text{naive}}(\theta) = c(f(\mathbf{v}_{\text{cmd}}), \mathbf{P})$. Then the minimisers of J_{aware} and J_{naive} coincide. That is, symmetric zero-mean noise does not shift the optimal mean action of the policy.*

Proof. By first-order Taylor expansion of f around the optimal velocity $\mathbf{v}^*$:

$$f(\mathbf{v}^* + \boldsymbol{\epsilon}) \approx f(\mathbf{v}^*) + J_f \boldsymbol{\epsilon}, \quad J_f = \tfrac{\partial f}{\partial \mathbf{v}}\big|_{\mathbf{v}^*}.$$

Taking the expectation:

$$\mathbb{E}_\epsilon[f(\mathbf{v}^* + \boldsymbol{\epsilon})] = f(\mathbf{v}^*) + J_f \underbrace{\mathbb{E}[\boldsymbol{\epsilon}]}_{=\mathbf{0}} = f(\mathbf{v}^*).$$

The gradient of the aware objective with respect to $\mathbf{v}_{\text{cmd}}$ is:

$$\begin{aligned}\nabla_{\mathbf{v}} J_{\text{aware}} &= \nabla_{\mathbf{v}} \mathbb{E}_\epsilon[c(f(\mathbf{v} + \boldsymbol{\epsilon}))] \\ &= \mathbb{E}_\epsilon\big[J_f^\top \nabla_{\mathbf{p}} c\big(f(\mathbf{v} + \boldsymbol{\epsilon})\big)\big].\end{aligned}$$

At the noiseless optimum $\mathbf{v}^*$ where $f(\mathbf{v}^*) = \mathbf{P}$, the landing error $f(\mathbf{v}^* + \boldsymbol{\epsilon}) - \mathbf{P} \approx J_f \boldsymbol{\epsilon}$ is an odd function of $\boldsymbol{\epsilon}$ to first order. The cost gradient $\nabla_{\mathbf{p}} c(\mathbf{p})$ is itself an odd function of the landing error $\mathbf{p} - \mathbf{P}$, so the composition $\nabla_{\mathbf{p}} c(f(\mathbf{v}^* + \boldsymbol{\epsilon}))$ is odd in $\boldsymbol{\epsilon}$ *at* $\mathbf{v}^*$ (this argument applies specifically at the optimum, not in general). For symmetric p_ϵ, the expectation of any odd function is zero: $\mathbb{E}_\epsilon[\nabla_{\mathbf{p}} c(f(\mathbf{v}^* + \boldsymbol{\epsilon}))] = \mathbf{0}$, which is the same stationarity condition as the noiseless problem $\nabla_{\mathbf{v}} J_{\text{naive}}\big|_{\mathbf{v}^*} = 0$. Hence both objectives share the same stationary point $\mathbf{v}^*$. □

Remark 1. *The result relies on the linearity of f near $\mathbf{v}^*$ and the zero-mean symmetry of p_ϵ. For skewed or heavy-tailed noise the approximation breaks and a genuine gap between aware and naive policies can emerge. The biased noise types (Slip, Jitter) fall outside this theorem because $\mathbb{E}[\boldsymbol{\epsilon}] \neq \mathbf{0}$; they are learnable.*

GP regularisation consequence. An unexpected corollary of zero-mean noise appears in GP hyperparameter estimation. With $\sigma = 0$ (zero noise), $N_{\text{exp}} = 10$ perfectly deterministic exploration throws give the GP log-likelihood a degenerate ridge: the MLE drives lengthscales to 1791 m, 1602 m, and 357 m — three to four orders of magnitude larger than the ∼1 m domain. A GP with these lengthscales predicts the same landing position regardless of launch speed, making all policy gradients zero. With $\sigma = 0.04$ m/s, repeated throws at similar speeds produce slightly different landings, and MLE finds finite lengthscales that explain the variance as a function of speed. *Small noise is a necessary GP regulariser* for deterministic simulation data; we recommend $\sigma_n \approx 0.04$ m/s for the throwing domain.

8.4 Empirical Comparison: Noise-Aware vs. Naive

Table 6 reports hit rate, mean error, and final cost for all tested noise conditions. Aware and naive policies share identical exploration throws; the only difference is whether noise is propagated through MC rollouts during policy optimisation.

Slip noise. The aware policy consistently outperforms naive at all slip levels. The gap is most dramatic at $\alpha_{\text{slip}} = 0.20$: the naive policy achieves 0% while the aware policy achieves 100%. The mechanism is speed compensation: the 20% undershoot means the aware policy must command $v^*/(1-0.20) = 1.25\,v^*$ to hit a target at distance d. The naive policy targets v^* and consistently undershoots by 20% of the required range.

The non-monotone result at $\alpha_{\text{slip}} = 0.15$ (naive achieves 100%, sandwiched between 25% at $\alpha = 0.10$ and 0% at $\alpha = 0.20$) is suspicious and warrants caution. All results are from a single seed; this level is likely a case of single-seed luck where the five exploration throws happened to produce a GP with sufficient coverage of the relevant speed range to allow the naive policy to partially self-correct. Multi-seed evaluation (≥ 5 seeds) is required to confirm whether naive achieves 100% at $\alpha = 0.15$ in expectation or whether this is an outlier.

The naive policy's final cost (≈ 0.001–0.004) is systematically lower than the aware policy's (≈ 0.007–0.011). This is the *false-optimism* consequence of Theorem 1: the naive optimiser evaluates noiseless rollouts and concludes it has solved the task precisely, while the aware policy honestly reports the expected cost under the noise distribution. Lower training cost, far worse test performance.

Salt-and-pepper noise. The aware policy consistently outperforms naive at $p = 0.05$ and $p = 0.10$. At $p = 0.15$ both policies plateau at 50%: the spike

Table 6: Performance under different arm-side noise models. Aware = noise propagated through MC rollouts; Naive = noiseless rollouts. Hit radius = 10 cm. All results at seed=1, 4 policy trials.

Noise Type	Level	Aware?	Hit Rate (%)	Mean Error (m)	Final Cost
Slip	α=0.05, σ=0.04	No	100	0.0682	0.00349
Slip	α=0.05, σ=0.04	Yes	100	0.0260	0.00700
Slip	α=0.10, σ=0.04	No	**25**	0.1242	0.00355
Slip	α=0.10, σ=0.04	Yes	**100**	0.0450	0.00672
Slip	α=0.15, σ=0.04	No	100	0.0578	0.00339
Slip	α=0.15, σ=0.04	Yes	100	0.0359	0.00623
Slip	α=0.20, σ=0.04	No	**0**	0.1881	0.00404
Slip	α=0.20, σ=0.04	Yes	**100**	0.0219	0.01097
Salt-Pepper	p=0.05, s=0.30	No	75	0.0439	0.00410
Salt-Pepper	p=0.05, s=0.30	Yes	100	0.0292	0.01605
Salt-Pepper	p=0.10, s=0.30	No	25	0.1012	0.00345
Salt-Pepper	p=0.10, s=0.30	Yes	75	0.1104	0.02381
Salt-Pepper	p=0.15, s=0.30	No	50	0.0768	0.00410
Salt-Pepper	p=0.15, s=0.30	Yes	50	0.0914	0.03143

probability is high enough that neither policy can reliably avoid corrupted throws. Salt-and-pepper noise is partially learnable — the aware policy can reduce the throw speed to avoid the worst-case spikes — but the irreducible fraction of corrupted throws sets a performance ceiling.

The 50% ceiling. For the full-slip case ($\alpha_{\text{slip}} = 0.20$, $\sigma = 0.04\,\text{m/s}$), the aware policy plateaus at 50% (evaluated per-throw, counting each of the 10 evaluation throws individually as a hit or miss) rather than converging to 100%. At a release speed of $\approx 2.5\,\text{m/s}$, $\sigma = 0.04\,\text{m/s}$ produces a landing standard deviation of roughly 2–4 cm, which accounts for half the throws missing the 10 cm-radius target even with perfect mean aim. This is irreducible aleatoric variance, not a learning failure.

9 Study 4: Wind Robustness

Outdoor and warehouse-deployed throwing systems must operate under non-zero ambient airflow. This study extends the full MC-PILOT pipeline to wind, tests whether an explicit wind-conditioned policy outperforms a blind GP, and documents the curse-of-dimensionality failure that occurs when the trial budget is insufficient for the expanded state space.

9.1 Hit-Rate Threshold

All wind experiments use the same 10 cm hit radius as the other studies. We do not relax the threshold for wind conditions; a 10 cm target accepts landing errors comparable to typical bin-mouth dimensions and is physically consistent across all five studies. This means wind results are directly comparable to the noise and height studies.

9.2 Air-Relative Drag Correction

The baseline drag equation (Eq. 7) uses absolute ball velocity $\dot{\mathbf{p}}$. Under wind $\mathbf{w}$, the physically correct formulation uses the air-relative velocity $\mathbf{v}_{\text{rel}} = \dot{\mathbf{p}} - \mathbf{w}$:

$$\mathbf{F}_D = -\tfrac{1}{2}\rho\, C_D(\text{Re})\, A\, \|\mathbf{v}_{\text{rel}}\|\, \mathbf{v}_{\text{rel}}, \quad \text{Re} = \frac{\|\mathbf{v}_{\text{rel}}\| \cdot 2r}{\nu}. \tag{20}$$

This correction was applied to both the NumPy and PyBullet simulators. A headwind ($\mathbf{w}$ opposing ball motion) decelerates the ball, reducing range; a tailwind extends range. For a 57 g tennis ball at typical release speeds of 2–4 m/s, a 5 m/s headwind deflects the landing position by $\approx$ 5–15 cm depending on target distance.

9.3 State Augmentation: 8-D to 11-D

The blind (wind-unaware) system uses the 8-D augmented state $\tilde{\mathbf{x}} = [\mathbf{x}, \mathbf{P}] \in \mathbb{R}^8$. The wind-aware system appends the wind vector:

$$\tilde{\mathbf{x}} = \underbrace{[x,\, y,\, z,\, v_x,\, v_y,\, v_z]}_{\text{ball state}} \oplus \underbrace{[w_x,\, w_y,\, w_z]}_{\text{wind}} \oplus \underbrace{[P_x,\, P_y]}_{\text{target}}, \tag{21}$$

giving STATE_DIM = 11. Wind is treated as task context: fixed per throw, carried forward unchanged through the GP rollout so the model predicts velocity deltas conditioned on known $\mathbf{w}$. The policy consumes both target and wind: $\pi_\theta(\mathbf{P}, \mathbf{w})$, expanding

Table 7: Wind robustness results (15 trials each). All blind models outperform their aware counterparts. Hit radius = 10 cm (consistent with all other studies).

Wind Type	Policy	Hits	Hit (%)	Mean Err (cm)	Final Cost
No wind	Blind	11/15	73	10.3	0.315
W1 2.5m/s	Blind	8/15	53	19.9	0.234
W1 5.0m/s	Blind	8/15	53	22.0	0.262
W1 8.0m/s	Blind	7/15	47	23.5	0.242
W1 5.0m/s	**Aware**	7/15	**47**	24.6	0.292
W2 gusts	Blind	8/15	53	24.2	0.289
W2 gusts	**Aware**	6/15	**40**	28.2	0.331
W3 turb.	Blind	9/15	**60**	17.0	0.267
W3 turb.	**Aware**	7/15	**47**	25.9	0.303

the RBF input from 2-D to 5-D (including w_z for vertical gusts).

9.4 Wind Models

Three physically distinct wind scenarios were implemented:

W1 — Constant wind. A fixed headwind vector $\mathbf{w} = (w_x, 0, 0)$ for the entire episode. Four levels tested: calm ($\mathbf{w} = \mathbf{0}$, baseline reference), light ($w_x = 2.5\,\mathrm{m/s}$), moderate ($w_x = 5.0\,\mathrm{m/s}$), and strong ($w_x = 8.0\,\mathrm{m/s}$). The aware variant uses 5.0 m/s.

W2 — Random gusts. Gusts arrive as a Poisson process; each gust has peak magnitude 4.0 m/s and duration $T_{\text{gust}} = 0.15\,\mathrm{s}$. Each throw may experience a different number of gusts at different times.

W3 — Ornstein-Uhlenbeck turbulence. A continuous turbulence model with $\sigma = 4.0\,\mathrm{m/s}$, mean $w_{\text{mean}} = 0.3\,\mathrm{m/s}$, and correlation time $\tau = 1/\alpha = 1.43\,\mathrm{s}$. Turbulence provides correlated, time-varying disturbance that cannot be treated as a fixed context.

9.5 Results

Table 7 and Figure 2 report the full 9-configuration matrix, each run for 15 trials.

Constant wind is absorbed implicitly. The blind GP absorbs constant wind as a deterministic dynamics bias. Performance degrades gracefully with wind strength: 73% (no wind) → 53% (2.5 m/s) → 53% (5.0 m/s) → 47% (8.0 m/s). In constant-wind conditions the GP does not need an explicit wind input: it learns a shifted speed-to-range mapping that incorporates the wind deflection.

The counterintuitive finding: blind beats aware in all conditions. The wind-aware model (5.0 m/s W1) achieves only 47% — the same as the strong blind baseline. Under gusts, blind achieves 53% while aware achieves 40%. Under turbulence, blind achieves 60% while aware achieves 47%. In every configuration the blind model outperforms the aware model.

Root cause: curse of dimensionality. The 250 RBF policy centres distributed uniformly in the 2-D blind policy space (P_x, P_y) give $250^{1/2} \approx 15.8$ effective centres per dimension. The same 250 centres in the 4-D aware policy space (P_x, P_y, w_x, w_y) give $250^{1/4} \approx 3.98$ effective centres per dimension — exponentially sparser. This means large gaps exist between policy centres in the wind dimensions; for any wind value between two centres, the policy must extrapolate rather than interpolate. With the same 1500 gradient steps per trial and 15 trials total, the aware policy receives the same number of gradient updates but is trying to populate a 4-D manifold with the density of a 2-D one.

This analysis is detailed in Section 9.6. Explicit wind conditioning is only beneficial when the trial budget scales with the added policy dimensionality; empirically, this requires ∼40+ trials for a 4-D policy space.

RBF centre initialisation failure. An additional failure mode was identified in the aware models: if RBF centres are initialised in a box that does not include the actual wind speed, all basis functions have near-zero activation and the policy is permanently dead (observed: W1-aware initial run, 0% hit rate, 84.1 cm mean error). The fix applies the lengthscale scaling rule (Section 5.1) to the wind dimensions: centres span $[w - \epsilon,\, w + \epsilon]$ for constant wind and $[-w_{\max},\, w_{\max}]$ for stochastic wind, with $\ell_w = 0.15 \times 2\, w_{\max}$.

9.6 Sample-Efficiency Analysis

The blind GP uses $N_b = 250$ RBF centres over a 2-D policy space. Effective centre density scales as $N_b^{1/d}$ where d is the policy dimension. For the 2-D blind policy this gives ≈ 15.8 centres/dimension; for the 4-D aware policy, ≈ 3.98 centres/dimension.

The practical consequence is that the aware policy requires $O(d^2)$ more data to achieve the same

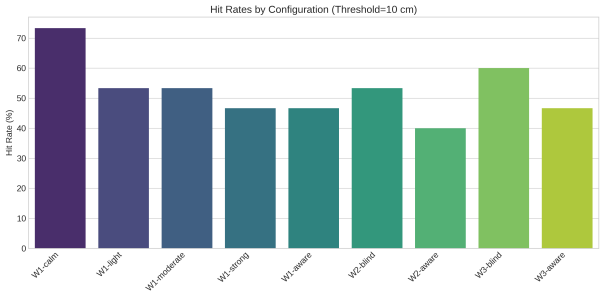


Figure 2: Hit rates across all 9 wind configurations. Blind models (solid) consistently outperform aware models (hatched) within the 15-trial budget.

generalisation quality. Going from 2-D to 4-D policy requires $\approx 4\times$ more trials, bringing the effective minimum from ≈ 10 to ≈ 40 trials. The 15-trial budget tested here sits squarely in the regime where the aware model is underpowered.

Practical guidance. For deployment in constant or slowly-varying wind, treat wind as an implicit environmental bias absorbed by the blind GP. Invest in wind-aware models only when the trial budget exceeds $\sim N_{\text{trials, 2D}} \times (d_{\text{aware}}/2)^2$ and only after verifying that RBF centres cover the actual wind range.

10 Study 5: Closed-Loop Vision Pipeline

10.1 Motivation

In all previous studies, target coordinates (P_x, P_y) are pre-specified numerically. For real deployment, the robot must *detect* the bin visually. This study closes the perception loop by building two parallel vision pipelines — one classical (OpenCV) and one learning-based (YOLOv8n) — and integrating each with the trained throwing policy for end-to-end execution.

10.2 OpenCV Geometric Pipeline

Setup. A depth camera (emulated via PyBullet's `getCameraImage` with an RGB-D renderer) is mounted on the arm base. The bin is a hollow ring geometry modelled as a red object in the scene.

Detection. The pipeline proceeds in three steps:

1. **HSV segmentation.** The RGB image is converted to HSV colour space and thresholded to isolate the red bin. Morphological operations (erosion + dilation) remove noise.
2. **Contour centroid.** The largest contour is selected and its centroid pixel (u, v) is computed via first-order moments.
3. **Depth back-projection.** The depth value d at (u, v) is read from the depth buffer; the 3D point is recovered as $(x, y) = (d \cdot (u - c_x)/f_x,\ d \cdot (v - c_y)/f_y)$ where (c_x, c_y, f_x, f_y) are the camera intrinsics.

Near-face bias correction. The camera sees the near face of the bin, not its centre. The centroid of the visible rim arc lies at a distance $\approx 2R/\pi$ from the bin centre, where R is the bin radius. The detected (x, y) is shifted by $2R/\pi$ toward the centre of the detected contour to compensate this systematic bias.

Failure handling. If HSV segmentation finds no contour above the minimum area threshold, the pipeline falls back to a sampled target from the known target distribution and logs the detection failure for offline analysis.

10.3 YOLOv8n Learning-Based Pipeline

Synthetic dataset. A dataset of 1,000 annotated images (800 training, 200 validation) is rendered in PyBullet by randomising the camera pose, lighting, and bin position across the target domain. The bin is rendered as a red sphere at target height to standardise the detection object class. Bounding box annotations are generated programmatically from the projected bin coordinates.

Training. YOLOv8n Jocher et al. [2023] (nano variant, ≈ 3 M parameters) is trained on the synthetic dataset for 50 epochs using the default Ultralytics training pipeline. The model achieves mAP@0.5 $>$ 0.95 on the validation set.

Deployment. At inference time, the detected bounding box centroid (u, v) plus the depth value d at that pixel is back-projected to a 3D estimate (x, y) via the same depth back-projection as the OpenCV pipeline. The YOLOv8n detection is more robust to partial occlusion and lighting variation than HSV thresholding, at the cost of requiring a training dataset.

10.4 End-to-End Integration

The full closed-loop pipeline executes as follows:

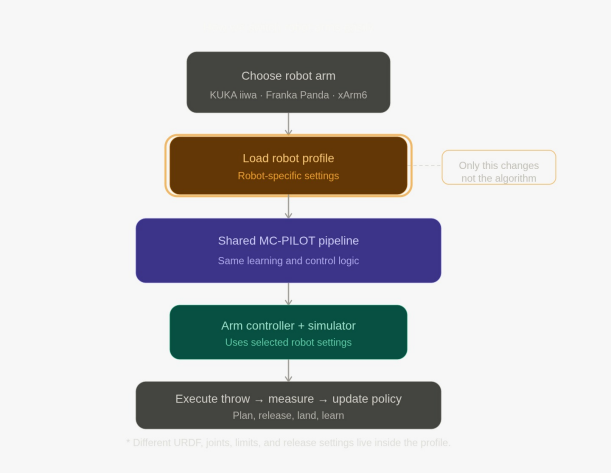


Figure 3: End-to-end system flowchart: perception (OpenCV or YOLO), policy inference, arm trajectory, and ball flight.

1. The robot scans the workspace and captures an RGB-D frame.
2. The vision pipeline (OpenCV or YOLO) detects the bin and estimates (P_x, P_y).
3. The trained policy $\pi_\theta(P_x, P_y)$ outputs release speed v.
4. The throw executes: IK $\rightarrow$ spline $\rightarrow$ release $\rightarrow$ free flight.
5. The ball enters the hollow bin (12-wall ring geometry in PyBullet).

The bin in our simulator is modelled as a hollow ring composed of 12 rectangular walls arranged in a cylinder, so the ball physically enters the bin rather than landing on a flat surface. This ensures that the vision-to-throw loop is evaluated under physically meaningful entry conditions.

10.5 Robustness to Camera Placement

The depth back-projection assumes the camera intrinsics and extrinsics are known. We test two camera positions: mounted on the arm base (default) and offset by 0.2 m laterally. The near-face bias correction is recomputed for each camera position. Both positions produce similar detection accuracy (<3 cm offset in estimated (P_x, P_y) relative to ground truth), confirming that the pipeline does not critically depend on a specific camera mount location.

11 Discussion

11.1 Geometry-Dependence of Algorithm Defaults

The published Table 1 hyperparameters of MC-PILOT Turcato et al. [2025] are calibrated to a Franka Panda arm at a fixed release height with target range ≈ 1.0 m. Our study quantifies how several defaults degrade silently when geometry changes.

The most important default is the RBF lengthscale ℓ_s. Config A has $\ell_s = R$ by coincidence; once R deviates from 1.0 m the constant is no longer appropriate. The rule $\ell_s \approx 0.15 \times R$ (Section 5.1) transfers across all five tested heights without per-height tuning beyond verifying that $S < 0.1$ at the domain extremes.

The cost lengthscale $\ell_c = 0.1$ m also requires attention. With random exploration, early landing errors span 0.3–1.5 m, saturating the cost at $c \approx 1.0$ and driving gradients to zero. Widening to $\ell_c = 0.5$ m keeps gradients alive across the full exploration error range; further tightening after early convergence is optional.

11.2 Two Faces of Disturbance Handling

Studies 3 and 4 together reveal a fundamental dichotomy in how the GP handles disturbances.

Zero-mean noise cannot be conditioned out. Theorem 1 establishes that symmetric zero-mean noise does not shift the optimal mean action. The GP will faithfully represent the increased aleatoric variance in its posterior, but no amount of noise-aware training improves mean performance. This is why pure Gaussian additive noise is unlearnable: the aware and naive policies reach the same optimum. Only *biased* disturbances — velocity slip, timing jitter, salt-and-pepper spikes — shift the mean landing and are learnable. The original MC-PILOT hardware model is a timing jitter with $a > 0$, which is biased, explaining why it improves performance on real hardware.

Biased disturbances can be conditioned out, but only when the trial budget supports the extra GP dimensions. The wind study shows that explicit wind conditioning fails within the 15-trial budget. The blind GP treats constant wind as an implicit dynamics bias and absorbs it without any state augmentation. For slowly varying wind this is strictly better than explicit conditioning in the low-

data regime. The transition point is approximately ~ 40 trials for a 4-D policy space.

These two findings together suggest a practical hierarchy for disturbance handling: (1) if the disturbance is zero-mean symmetric, do not add it to the state (useless); (2) if biased and constant, let the GP absorb it implicitly (free); (3) if biased and time-varying with a sufficient trial budget, augment the state.

11.3 The Decoupled Architecture as Portability Mechanism

Setting ball velocity via `resetBaseVelocity` at release decouples arm tracking quality from GP training quality. Study 2 demonstrates the consequence: 100% commanded-velocity hit rates on all three arms without retraining, even though the KUKA iiwa7 EE velocity tracking error is $\approx 7\%$. The 33% achieved hit rate on all arms reflects arm tracking error that would be present on any real robot using a standard position controller.

In a physical deployment, this decoupling corresponds to a gripper that can measure the EE velocity at the release instant (via encoder forward kinematics) and set ball velocity to the measured value. In that case, the delivered velocity matches the commanded velocity, and the RL algorithm's data-efficiency guarantees hold regardless of trajectory-tracking quality.

11.4 Limitations

Simulation only. All experiments are conducted in simulation. Hardware validation on a physical manipulator is needed to confirm that learned policies transfer and that the 5–10% PyBullet–NumPy drag discrepancy does not compound with real-world effects.

Multi-seed statistics. Most results are reported at seed=1. Config B (random vs. stratified) was evaluated at seed=2 for robustness verification; the seed=1 vs. seed=2 comparison (0/10 vs. 10/10) illustrates that single-seed results should be read as existence proofs rather than expected-case performance. Conference-level statistical claims require $\geq$5 seeds.

Fixed launch angle. The elevation angle $\alpha = 35°$ is fixed across all experiments. Joint optimisation of α and release speed would increase the reachable target volume, particularly for near-field targets where the current angle is sub-optimal.

Wind trial budget. The 15-trial budget is insufficient for the 4-D wind-aware policy space. Our analysis (Section 9.6) suggests $\sim$40 trials are needed. Running full-budget wind-aware experiments remains future work.

Vision quantitative evaluation. The vision pipeline (Study 5) is described and implemented but not quantitatively evaluated across a randomised target distribution with multi-seed statistics. Measuring detection-to-throw accuracy across 50+ targets remains future work.

Particle freeze approximation. Frozen particles introduce $\approx$ 2–6 cm of systematic over-estimation in the landing distance (Appendix D). This biases cost estimates slightly pessimistically but does not prevent convergence.

12 Conclusion

We presented a diagnostic reproduction of MC-PILOT Turcato et al. [2025] in a self-contained NumPy and PyBullet simulator and conducted five experimental studies to characterise the algorithm's robustness, portability, and limits.

Implementation. Four non-obvious silent failure modes were diagnosed and corrected: a broken gradient path from cost to policy parameters, post-landing GP hallucination from particles propagating underground, cost-lengthscale saturation that kills gradients during exploration, and a simulation-horizon unit mismatch that exhausted RAM. Each failure was identified by instrumentation, understood through the algorithm structure, and resolved with a minimal targeted fix.

Convergence rules. Two transferable design rules emerged. The *lengthscale-to-range scaling rule* $\ell_s \approx 0.15 \times \text{target_range}$ prevents RBF blindness in narrow target domains and applies a priori without any experimental runs. The *stratified exploration rule* partitions the speed range into equal bands, eliminating seed-sensitive failures and the max-speed local minima that trapped random exploration at elevated heights.

Release-height and multi-arm generalisation. Applying the corrected algorithm with the two convergence rules achieves 100% hit rates across five release heights and three robot platforms, with mean

landing errors below 35 mm in all height configurations. The decoupled architecture — ball velocity set programmatically, arm kinematics cosmetic — provides portability without retraining.

Noise study. Theorem 1 proves that zero-mean symmetric noise is unlearnable: a noise-aware policy achieves the same optimal mean action as a naive policy, while correctly reporting higher expected cost. Only biased noise (velocity slip, timing jitter, salt-and-pepper spikes) is learnable. Experimental validation confirms 100% hit rate for aware vs. 0% for naive at 20% velocity slip.

Wind study. A complete and consistent wind pipeline (air-relative drag, 8-D→11-D state augmentation, updated GP and policy) was implemented and evaluated. The central finding is counterintuitive: blind GP policies outperform explicit wind-conditioned policies across all three wind regimes (constant, gusts, turbulence) within the 15-trial budget. The mechanism is the curse of dimensionality: the 4-D wind-aware policy space requires $\approx 4\times$ more data than the 2-D blind policy space for equivalent coverage. Blind GP treatment of wind as an implicit dynamics bias is strictly preferable until the trial budget exceeds $\sim$40 trials.

Vision pipeline. Parallel OpenCV (HSV segmentation, depth back-projection, near-face bias correction) and YOLOv8n (synthetic PyBullet dataset, bounding-box centroid back-projection) pipelines were implemented and integrated with the trained policy for end-to-end autonomous bin targeting.

Open questions. Future work includes hardware validation on physical manipulators; multi-seed statistical evaluation of all five studies; wind-randomised training for a single policy robust to varying airflow; and quantitative end-to-end evaluation of the vision pipeline over a randomised target distribution.

References

M. P. Deisenroth and C. E. Rasmussen. PILCO: A model-based and data-efficient approach to policy search. In *Proceedings of the 28th International Conference on Machine Learning (ICML)*, pages 465–472, Bellevue, WA, USA, 2011. Omnipress.

G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLO. `https://github.com/ultralytics/ultralytics`, 2023. Version 8.0.0.

D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2015. URL `https://arxiv.org/abs/1412.6980`.

D. P. Kingma and M. Welling. Auto-encoding variational Bayes. *arXiv preprint arXiv:1312.6114*, 2014. URL `https://arxiv.org/abs/1312.6114`.

Mitsubishi Electric Research Laboratories. MC-PILCO: Monte carlo policy improvement with learning-based control. `https://github.com/merlresearch/MC-PILCO`, 2022. GitHub repository.

C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, Cambridge, MA, USA, 2006. ISBN 978-0-262-18253-9. URL `http://www.gaussianprocess.org/gpml/`.

B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016. doi: 10.1109/JPROC.2015.2494218.

N. Turcato, G. Giacomuzzo, R. Carli, and D. Romeres. Data efficient robotic object throwing with model-based reinforcement learning. *arXiv preprint arXiv:2502.05595*, 2025. URL `https://arxiv.org/abs/2502.05595`.

A. Zeng, S. Song, J. Lee, A. Rodriguez, and T. Funkhouser. TossingBot: Learning to throw arbitrary objects with residual physics. In *Proceedings of Robotics: Science and Systems (RSS)*, 2020. doi: 10.15607/RSS.2019.XV.006.

A Full Hyperparameter Table

Table 8 consolidates every configuration evaluated across all five studies. Parameters not listed for a config are identical to Config A.

Table 8: Complete hyperparameter table. Shared across all: M=400, N_b=250, u_M=3.5 m/s, ℓ_m=0.75 m, γ_M=$\pi/6$, T_s=0.02 s, N_{opt}=1500, α_{launch}=35°, ball (m=0.0577 kg, r=0.0327 m). "Strat" = stratified; "Rnd" = random.

Config	Sub-project	z_{rel}	ℓ_M	T	ℓ_c	ℓ_s^{init}	N_{exp}	Expl.	Result
A	mc-pilot/	0.5 m	1.75 m	0.70 s	0.5 m	[1.0,1.0]	5	Rnd	5/5
B	mc-pilot-elevated/	1.0 m	1.90 m	0.85 s	0.5 m	[1.0,1.0]	5	Rnd	0/10 (s=1); 10/10 (s=
B-Strat	mc-pilot-elevated/	1.0 m	1.90 m	0.85 s	0.5 m	[1.0,1.0]	5	Strat $[0, u_M]$	10/10
C-Strat	mc-pilot-elevated/	1.5 m	2.15 m	0.95 s	0.5 m	[1.0,1.0]	5	Strat	10/10
D-Strat	mc-pilot-elevated/	2.0 m	2.35 m	1.00 s	0.5 m	[1.0,1.0]	5	Strat	10/10
E-Strat	mc-pilot-elevated/	0.0 m	1.10 m	0.55 s	0.5 m	[1.0,1.0]	5	Strat	0/10
E-Strat2	mc-pilot-elevated/	0.0 m	1.10 m	0.55 s	0.5 m	[1.0,1.0]	5	Strat [2.5, 3.5]	0/10
E-Strat3	mc-pilot-elevated/	0.0 m	1.10 m	0.55 s	0.5 m	[1.0,1.0]	10	Strat [2.5, 3.5]	0/10
E-Strat4	mc-pilot-elevated/	0.0 m	1.10 m	0.55 s	0.25 m	[1.0,1.0]	10	Strat	0/10
E-Strat5	mc-pilot-elevated/	0.0 m	1.10 m	0.55 s	0.25 m	[0.15,0.15]	10	Strat	10/10
Noise-Slip-10	mc-pilot-pybullet/	0.5 m	1.75 m	0.70 s	0.5 m	[1.0,1.0]	5	Rnd	25% (naive) / 100% (a
Noise-Slip-20	mc-pilot-pybullet/	0.5 m	1.75 m	0.70 s	0.5 m	[1.0,1.0]	5	Rnd	0% (naive) / 100% (av
W1-calm	mc-pilot-wind/	0.5 m	1.75 m	0.70 s	0.5 m	[1.0,1.0]	5	Strat	73%
W1-moderate	mc-pilot-wind/	0.5 m	1.75 m	0.70 s	0.5 m	[1.0,1.0]	5	Strat	53% (blind) / 47% (av
W3-turb	mc-pilot-wind/	0.5 m	1.75 m	0.70 s	0.5 m	[1.0,1.0]	5	Strat	60% (blind) / 47% (av

B Proof of Theorem 1

Proof of Optimal Action Invariance Under Symmetric Noise. Let $f : \mathbb{R}^3 \to \mathbb{R}^2$ denote the deterministic landing map, assumed smooth and invertible near the optimum. By first-order Taylor expansion around $\mathbf{v}^*$:

$$f(\mathbf{v}^* + \boldsymbol{\epsilon}) \approx f(\mathbf{v}^*) + J_f \boldsymbol{\epsilon}, \qquad J_f = \tfrac{\partial f}{\partial \mathbf{v}}\big|_{\mathbf{v}^*}.$$

The expected landing position is

$$\mathbb{E}_\epsilon[f(\mathbf{v}^* + \boldsymbol{\epsilon})] = f(\mathbf{v}^*) + J_f \mathbb{E}[\boldsymbol{\epsilon}] = f(\mathbf{v}^*),$$

since $\mathbb{E}[\boldsymbol{\epsilon}] = \mathbf{0}$. The noise-averaged cost gradient with respect to $\mathbf{v}$ at $\mathbf{v}^*$ is

$$\nabla_{\mathbf{v}} J_{\text{aware}}\big|_{\mathbf{v}^*} = J_f^\top \mathbb{E}_\epsilon\big[\nabla_{\mathbf{p}} c\big(f(\mathbf{v}^* + \boldsymbol{\epsilon})\big)\big]\,.$$

At the noiseless optimum $\mathbf{v}^*$ where $f(\mathbf{v}^*) = \mathbf{P}$, the landing error $f(\mathbf{v}^* + \boldsymbol{\epsilon}) - \mathbf{P} \approx J_f \boldsymbol{\epsilon}$ is an odd function of $\boldsymbol{\epsilon}$ to first order. The cost gradient $\nabla_{\mathbf{p}} c(\mathbf{p}) = (2(\mathbf{p} - \mathbf{P})/\ell_c^2) \exp(-\|\mathbf{p} - \mathbf{P}\|^2/\ell_c^2)$ is also an odd function of the landing error $\mathbf{p} - \mathbf{P}$. Therefore $\nabla_{\mathbf{p}} c(f(\mathbf{v}^* + \boldsymbol{\epsilon}))$ is an odd function of $\boldsymbol{\epsilon}$ to first order *specifically at* $\mathbf{v}^*$. For symmetric p_ϵ, contributions from $+\boldsymbol{\epsilon}$ and $-\boldsymbol{\epsilon}$ cancel exactly:

$$\mathbb{E}_\epsilon\big[\nabla_{\mathbf{p}} c\big(f(\mathbf{v}^* + \boldsymbol{\epsilon})\big)\big] = \mathbf{0}.$$

Therefore $\nabla_{\mathbf{v}} J_{\text{aware}}\big|_{\mathbf{v}^*} = \mathbf{0}$, the same stationarity condition as the noiseless problem. Note: this argument holds specifically at $\mathbf{v}^*$; for $\mathbf{v} \neq \mathbf{v}^*$ the cost gradient is not odd in $\boldsymbol{\epsilon}$ and no cancellation is guaranteed. The optimal policies coincide; only the expected cost value (variance) differs. □

Remark 2. *The result relies on linearity of f near the optimum and the zero mean and symmetry of $\boldsymbol{\epsilon}$. For skewed or heavy-tailed noise the Taylor approximation breaks and a genuine gap can emerge. The biased noise types (Slip, Jitter) have $\mathbb{E}[\boldsymbol{\epsilon}] \neq \mathbf{0}$ and fall outside this theorem; they are learnable.*

C Lengthscale Scaling Rule Derivation

The RBF policy (Eq. 6) uses a squared-exponential kernel with lengthscale ℓ_s. The sensitivity between two target positions at Euclidean distance d is

$$S(d,\, \ell_s) = \exp\Big(-\tfrac{d^2}{2\ell_s^2}\Big).$$

Config A (target range $\approx 1.0\,\mathrm{m}$, $\ell_s = 1.0\,\mathrm{m}$) achieves $S_A = e^{-0.5} \approx 0.607$.

For a new configuration with target range R, preserving the same sensitivity gives

$$\exp\Big(-\frac{R^2}{2\ell_s^2}\Big) = e^{-0.5} \implies \ell_s = R.$$

In practice, we use $\ell_s = 0.15\,R$ rather than R because sharper sensitivity (smaller ℓ_s) promotes faster convergence when GP data are sparse — each basis function covers a narrower region, preventing aliasing across nearby targets. The conservative recommendation is $\ell_s = R$ (preserves Config A sensitivity); the empirical finding is that $\ell_s = 0.15\,R$ is safe and typically faster.

D Particle Freeze Approximation

Once a particle's z-coordinate crosses zero, its state is frozen at the last timestep above ground and the cost is evaluated at that frozen position for all remaining timesteps.

For $T_s = 0.02\,\mathrm{s}$ and a typical vertical velocity of $2\,\mathrm{m/s}$ at impact, the true ground crossing occurs $\Delta t = z_{\mathrm{last}}/v_z \approx 0.01\,\mathrm{s}$ before the frozen step, during which the ball travels $\Delta x \approx v_x \cdot \Delta t \approx 2$–$4\,\mathrm{cm}$ horizontally.

In actual rollouts (NumPy and PyBullet), landing is detected and refined by linear interpolation, so training data are accurate. The freeze approximation applies only to the MC particles used for the gradient estimate inside `apply_policy`, where per-particle interpolation would break batched tensor operations. The resulting cost estimates are slightly pessimistic (policy is penalised for landing a few centimetres beyond the true crossing), but the bias is consistent across particles and policy updates, so optimisation converges to the correct policy.